

Proving what a second-hand computer is

An evidence-first architecture for IT asset disposition

*Assembled from paper/sections/ – edit those files, not this one. Drafting notes are excluded. Regenerate with
python paper/assemble.py.*

Contents

Contents	1
Abstract	5
1. Introduction	5
1.1 The thesis	6
1.2 The system	6
1.3 Contributions	6
1.4 How to read this	6
2. Requirements and constraints	7
2.1 The floor is not a network	7
2.2 The operator has a screwdriver in one hand	7
2.3 The machine is somebody else's, and it is evidence	7
2.4 The output is a legal document	7
2.5 Two constraints that are not about the machine	7
2.6 What this rules out	8
3. System overview	8
3.1 The components	8
3.2 The trust boundary	9
3.3 The direction of travel	9
3.4 Deployment, and the hazard in it	9
3.5 What each part is made of	9
4. Layer 1: the inventory model	10
4.1 The hierarchy, and the one deliberate exception	10
4.2 Expected against actual	10
4.3 Stock as a derived balance	10
4.4 Ownership, and the lesson that generalises	11

4.5 Append-only history	11
5. Layer 2: offline-first capture	11
5.1 The architecture	11
5.2 Four faults in the first week, three of them silent	11
5.3 Scanning: the decision, and what a warehouse did to it	12
5.4 The queue that had to survive a power cut	12
5.5 Two instrumentation faults with the same shape	13
6. Layer 3: the audit station	13
6.1 The base, and the constraint that chose it	13
6.2 Seven attempts to make a browser fill a screen	14
6.3 Who owns a keypress	14
6.4 The station's hardware is not the machine's	14
6.5 Accountability at the bench	15
6.6 How code reaches a warehouse	15
7. Layer 4: deep inspection	15
7.1 The constraint, stated as a capability	15
7.2 What a powered-off Windows will tell you	16
7.3 Three mechanical traps, and the one that got through	16
7.4 Locks, and the promotion rule	16
7.5 Drive health: never "Unknown"	17
7.6 Reading the installed operating system	17
8. Layer 5: erasure and certification	18
8.1 A ladder, not a technique	18
8.2 Seven things that were wrong, and one of them was already on paper	18
8.3 Making the document tamper-evident	19
8.4 What making it public cost	19
8.5 Keys	19
8.6 What the certificate refuses to say	19
9. Layer 6: functional testing, and the human as an instrument	20
9.1 The principle	20
9.2 Could-not-run is not failed	20
9.3 The seven tests	20
9.4 The keyboard test, which took five attempts	21
9.5 Why this layer ships by file copy	21

10. Layer 7: the back office	21
10.1 Permissions, inverted	21
10.2 Deleting a user is not an option	22
10.3 A dashboard that disagreed with itself	22
10.4 Reporting	22
10.5 Consistency as a defect detector	23
11. The knowability framework	23
11.1 Four tiers	23
11.2 The worked example	23
11.3 What was built instead	24
11.4 The general form	25
12. Evidence integrity: the false absence	25
12.1 The class	25
12.2 What was catalogued	25
12.3 Why the tests did not catch them	26
12.4 The recurrence finding	26
12.5 The response, and what cannot be claimed	27
13. The discipline of deletion	27
13.1 What each one got wrong	27
13.2 Removing things safely	28
13.3 The argument	28
13.4 The honest residue	28
14. Engineering practice	29
14.1 Design records	29
14.2 Testing something that erases drives	29
14.3 CI, and a list that stops covering things	29
14.4 Two migration hazards	30
14.5 Where the practice is weak	30
15. Results	30
15.1 What was built	30
15.2 What runs	31
15.3 What is proven, and how	31
15.4 The defect result	32

15.5 The negative results	32
16. Limitations and threats to validity	32
16.1 One site, one business, one engineer	32
16.2 The defect catalogue classifies its own author's work	33
16.3 What was never measured	33
16.4 Gaps in the system itself	33
16.5 Two decisions that are the owner's, not ours	33
16.6 Scope of the knowability framework	34
17. Lessons for building evidence-producing systems	34
17.1 Before writing a probe	34
17.2 While designing the vocabulary	34
17.3 While writing tests	34
17.4 While shipping	35
17.5 About the organisation, not the code	35
17.6 The one we would tell someone first	35
18. Related work	36
18.1 Media sanitisation and asset disposition	36
18.2 Offline-first and local-first systems	36
18.3 Dependability and error handling	36
18.4 Alarm design, inverted	36
18.5 What we did not find	37
19. Conclusion	37

Abstract

An IT asset disposition business resells second-hand computers and issues two documents about each one: a specification, and a certificate stating that the previous owner's data has been destroyed. Both are claims about a machine the business does not control, made once by a technician, about hardware that is then sold. Nothing in normal operation reveals when such a claim is wrong.

We report the design and construction of a complete production system for this job — 535 commits and roughly 101,000 lines over 78 days by one engineer — comprising an offline-first inventory platform, a bootable audit appliance that inspects and erases a customer's machine without starting its operating system, an inspection layer that reads a powered-off Windows from its registry hives, signed and chained erasure certificates, and a functional-test module that records a technician's judgement as the measurement it is. We describe each layer, what was rejected in building it, and what remains unproven.

Three findings generalise. First, claims about external state fall into four **knowability tiers** — measured, recovered from an artefact, perishable, or withheld by a third party — and the tier determines whether code, a process change, or nothing will produce an answer; the commercially most valuable question in this domain sits in the fourth tier, and we report how it was answered by observation instead of deduction. Second, we catalogue 32 call sites exhibiting a defect we name the **false absence**: a failed observation reported as a confirmed negative. All 32 failed towards the reassuring answer; 18 sat on code paths with existing tests, which caught none. Third, and least comfortable: naming that class in a commit, documenting it in the affected code and defending it with tests did not prevent the same author writing eight fresh instances within three weeks.

We also report three deliberate feature removals, and argue that the useful measure of a system under development is how much of it is load-bearing.

1. Introduction

When an organisation replaces its computers, the old ones do not stop existing. They go to an IT asset disposition (ITAD) company, which has two obligations that pull in opposite directions. It must **destroy the data** those machines hold, and prove it. And it must **describe each machine accurately** to whoever buys it next, which means examining it closely enough to say what it is.

Both obligations produce documents. An erasure certificate is a legal instrument: the organisation that owned the data remains accountable for it under data-protection law, and the certificate is the evidence it relies on. A specification is a commercial one: a buyer paying for 16 GB of memory and a healthy drive is entitled to get them.

Both documents are **claims about a machine the issuer does not control**, made once, briefly, by a technician with a screwdriver in one hand, about hardware that is then packed into a pallet and sold. And this is the difficulty that shapes everything in this paper:

Nothing in the normal operation of such a system tells you when it is wrong.

A machine reported as having no encryption, when in fact the probe that looked for encryption failed, produces a document that reads exactly like a correct one. It passes review. It satisfies the buyer at the moment of sale. The error surfaces weeks later, in somebody else's hands, if it surfaces at all. There is no exception, no alert, no failing test, and no complaint — because the wrong answer is the reassuring one.

Conventional software engineering has weak defences here. Tests assert that code behaves correctly when its inputs behave correctly. Monitoring catches things that crash. Code review reads the path the author was thinking about. None of these reliably catch a probe that quietly returns *nothing found* because it could not look.

1.1 The thesis

This paper describes a complete production system built for a working ITAD business, and argues one idea that runs through every layer of it:

Every output of this system is a claim about a machine the system does not control. Its entire value is how honestly it separates what it has established from what it has merely assumed.

That is not a slogan bolted on afterwards. It is the design constraint that produced the offline queue's visible-failure rule, the audit station's refusal to report its own hardware as the machine's, the wipe engine's insistence on reading the drive back, the three-valued verdicts in lock detection and functional testing, and — at the limit — the decision that certain questions about a machine have **no offline answer at all** and must be reported as unanswered rather than guessed.

1.2 The system

Built	9 July – 24 September 2026, 78 days
Commits	535
Code	~101,000 lines across four deployable components
Domain entities	23, over 62 schema migrations
Test files	96 (46 server-side, 50 for the audit station)
Engineers	1

Four components: a **NestJS API** with Postgres, a **Next.js web application**, a **PowerSync** service giving the warehouse floor a local database that survives losing the network, and — the unusual one — a **bootable USB audit station** that boots a customer's machine into Linux, examines it without ever running its operating system, erases it, and reports back.

1.3 Contributions

1. **A complete architecture for evidence-producing asset disposition**, described layer by layer (§§4–10), including the parts that are ordinarily left out of systems papers: the boot chain, the kiosk hardening, and how code reaches a USB stick in a warehouse.
2. **A four-tier framework for machine knowability** (§11) — what a powered-off computer can be made to tell you about itself, where inference has to replace measurement, and where the answer lives in somebody else's cloud and cannot be obtained at all. We work it through Microsoft Autopilot enrolment, where we reached a hard limit and documented it rather than guessing past it.
3. **A defect class named, measured and catalogued** (§12): the *false absence*, a failed observation reported as a confirmed negative. 32 call sites across this codebase, every one failing towards the reassuring answer — and the uncomfortable result that naming it in a commit, documenting it in the code and defending it with tests did not stop eight fresh instances being written within three weeks.
4. **An account of what was deliberately removed** (§13). Three working, shipped modules were deleted because they modelled a business that does not exist. We argue this is why the system stayed small enough to keep changing.

1.4 How to read this

Part I is the problem and the shape of the system. Part II walks the layers from the data model outward to the machine on the bench. Part III is what the building taught — the findings that generalise past this business. Part

IV reports what the system does in production, what is still unproven, and what we would tell anyone building something similar.

A reader who wants only the engineering findings can read §§11–14 and skip the rest. A reader who wants only the architecture can read Part II. The failures are kept in throughout, in the sections where they belong, because a paper that reports only what worked is a brochure.

2. Requirements and constraints

The shape of this system is almost entirely explained by four facts about where it runs and what it is for. None of them are software requirements in the usual sense; all four eliminate designs that would otherwise be obvious.

2.1 The floor is not a network

Machines are received in a loading bay, examined in a back room, and stacked on pallets in a warehouse. Wi-Fi reaches some of those places some of the time. A system that stops accepting work when the connection drops is a system that technicians route around, usually onto paper, and the paper never reaches the database.

R1. Capture must work with no connection at all, and reconcile later. This forces a local database on the client and an upload queue — and, as §5 describes, the queue then becomes the single most dangerous component in the system, because its failure mode is losing somebody's work silently.

2.2 The operator has a screwdriver in one hand

The person using the audit station is a warehouse technician, not a system administrator. They are standing at a bench, often with the machine open. They will not read a manual, mount a filesystem, or type a command. If the tool requires a decision they are not equipped to make, they will pick whichever option lets them continue.

R2. The station must run without being typed at, boot straight into its interface, and never present a question whose wrong answer is invisible. This requirement drove a long sequence of work — autorun, a desktop launcher, a kiosk session, full-screen enforcement — described in §6.

2.3 The machine is somebody else's, and it is evidence

The subject of every examination is a computer that belonged to a customer, still holds their data, and may have to be returned or accounted for. Until the moment an erasure is deliberately authorised, nothing about it may change.

R3. Read-only by default. The customer's volumes are mounted read-only. The registry is read out of the hive files offline; the machine's own Windows is never booted. **R4. The station's hardware is not the subject's.** This sounds trivial and is not: the very first version of the audit tool recorded the USB stick it had booted from as the machine's hard drive (§6.4), and the same confusion recurs in OS detection and in the wipe engine's drive selection, where the consequence is erasing the wrong disk.

2.4 The output is a legal document

An erasure certificate is relied upon by the organisation that owned the data to discharge an obligation it cannot delegate. A specification is relied upon by a buyer paying money. Both are produced from automated probes of unfamiliar hardware, in a single pass, by a non-expert.

R5. Every claim must be separable into what was established and what was assumed, and that separation has to survive all the way to the sentence a person reads. A certificate that cannot express *we could not check this* will express *this is fine* instead — which is the defect class §12 is about.

2.5 Two constraints that are not about the machine

R6. The station carries credentials, on a stick that can be lost. It needs network configuration and a server account to upload what it finds. Its configuration file has therefore never been in the repository, and a standing rule forbids reading or printing its values anywhere — presence and format only. The account it uses is a restricted station account, never an administrator.

R7. Code has to reach a stick in a warehouse. There is no deployment pipeline to a USB device on a bench. Getting a change onto the stations is a physical act, and the cost of that act differs by an order of magnitude depending on which file changed (§6.6). This constraint shaped real design decisions — including, in one case, shipping a Debian package by file copy rather than rebuilding a filesystem image.

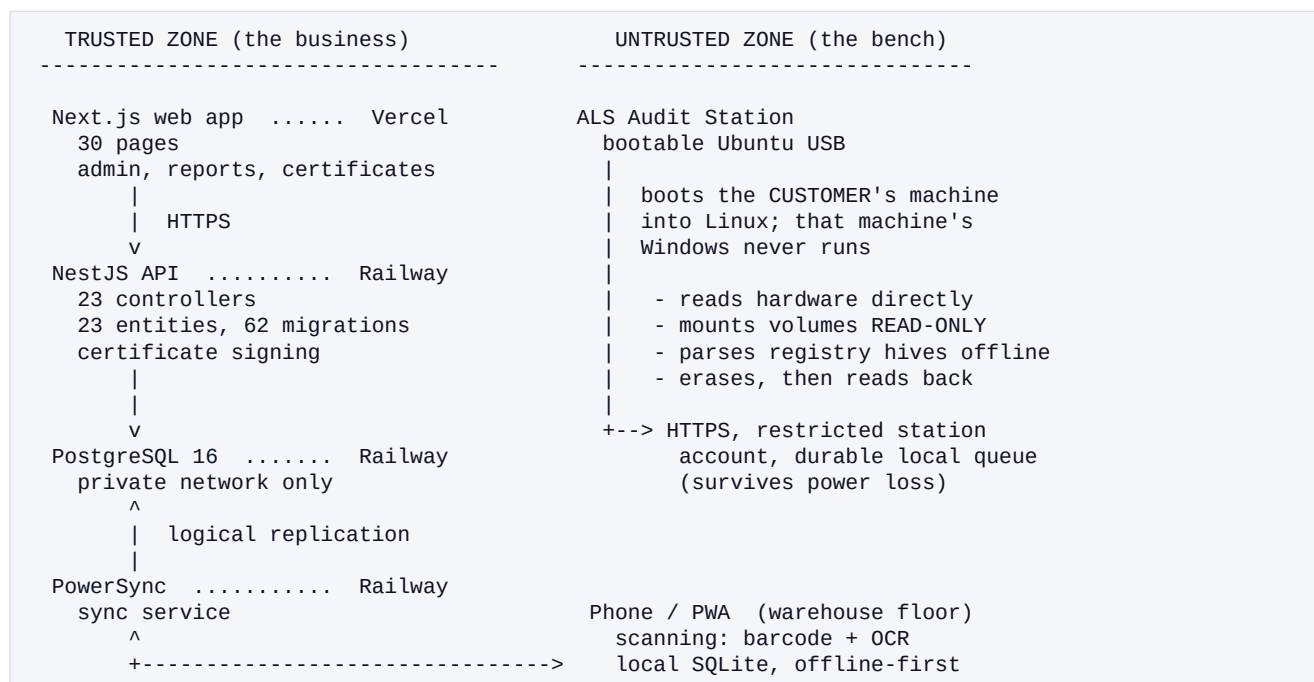
2.6 What this rules out

Obvious design	Why it fails here
Web app on the technician's phone, server-side truth	R1: no connection in the bay
Agent installed into the customer's Windows	R3: booting their OS changes it; many arrive locked or encrypted
Cloud API for device status	R3 and §11: the machine is offline and the answer is behind somebody else's authentication
Two-valued results (present / absent)	R5: cannot express a probe that failed
Continuous deployment to the stations	R7: they are USB sticks in a warehouse

3. System overview

Four deployable components, two trust zones, and one direction of travel: evidence is produced at the bench and flows inward to a record that is never edited to match it.

3.1 The components



The **web application** is the office: receiving, the inventory hierarchy, pallets, sales status, reporting, user administration, and the certificates. The **API** holds all business rules and is the only writer to the database. **PowerSync** gives the phone a local SQLite database that reconciles when a connection returns. The **audit station** is a USB stick.

3.2 The trust boundary

The right-hand side of the diagram is the interesting one. The audit station runs on hardware nobody in the business owns, on a machine that may be encrypted, locked, broken, or carrying somebody else's data. It holds credentials. It can be physically lost.

Three rules follow, and each is enforced in code rather than by convention:

1. **It writes nothing to the subject** unless an erasure has been explicitly authorised for that machine (R3).
2. **It authenticates as a restricted station account**, never an administrator, so a lost stick cannot administer the system (R6).
3. **Everything it reports is attributed to the station**, with the operator who was signed in recorded alongside it, so a wrong claim can be traced to a machine and a person rather than appearing as an anonymous fact.

3.3 The direction of travel

The system has one invariant that is worth stating on its own, because several later decisions are consequences of it:

Evidence flows inward. The record is never edited to agree with it.

An audit produces a profile. A wipe produces a verdict. A functional test produces a result. These accumulate on the asset as *findings with provenance* — which station, which operator, which run — and the asset's status is derived from them. When a later audit contradicts an earlier one, both are kept. When a finding is one the system could not establish, that is itself recorded (§7.3), rather than being resolved into a clean value on the way in.

This is why the data model in §4 separates an asset from its audits, why certificates are signed artefacts rather than rendered views, and why the activity log is append-only.

3.4 Deployment, and the hazard in it

The API and web app deploy on push to master; Railway builds the API, Vercel builds the web app. The audit station does not deploy — it is synced to a USB stick by hand (§6.6).

One deployment detail is load-bearing enough to belong in the architecture rather than an appendix. TypeORM issues a SELECT naming **every mapped column**. If a deploy ships an entity carrying a new column before the migration that creates it, every query against that table fails until the migration runs. This happened once, taking three tables down in the gap between deploy and migrate. The fix is that migrations run as a **pre-deploy command** on the newly built image, while the previous version is still serving traffic: if the migration fails the deploy aborts and the old version keeps running. Code and schema move together or not at all.

3.5 What each part is made of

Component	Language / stack	Size	Tests
API	TypeScript, NestJS, TypeORM, Postgres 16	~34,500 lines	46 spec files
Web	TypeScript, Next.js 15, React	~24,800 lines	—
Audit station	Bash, Python, HTML/JS on Ubuntu	~41,900 lines	50 test scripts
Sync	PowerSync, self-hosted	config + sync rules	—

The station being the largest single component is not an accident of style. It is where the system touches hardware it does not control, and §12 shows it is also where 28 of 32 catalogued defects of the most dangerous

class were found.

4. Layer 1: the inventory model

The first design mistake available in this domain is to believe that a warehouse contains one kind of thing. It contains three, and they need three different models.

Tier	Example	Identity	Counted by
Serialised asset	A laptop	Its own serial number	One row per physical device
Pallet line	Forty identical 24-inch monitors	The variant, not the unit	Quantity per variant
Consumable	Caddies, cables, screws	The stock line	A running balance

Forcing monitors into the asset table means inventing serial numbers for devices nobody will ever look up individually. Forcing cables in means forty rows for one box. Both were considered and rejected early, and the three-tier split has survived every subsequent redesign.

The system is honest about what this costs: a pallet line **cannot tell you which physical monitor is which**, and that is recorded as the correct model for the business rather than a limitation awaiting a fix.

4.1 The hierarchy, and the one deliberate exception

Stock arrives as a **purchase lot** — a pallet or van-load bought from a supplier — which contains devices, each of which has hardware. The interface follows that shape: **Lot** → **Asset** → **Hardware**, with a breadcrumb trail and Total / Audited / Pending counts on each lot card, so the state of a lot is legible without opening it.

One screen deliberately does not follow the hierarchy. The global assets page was kept as a **search**, because *what is in this lot* and *where is this serial number* are different questions and forcing the second through the first makes it slow and annoying. Consistency was traded for fitness, knowingly.

4.2 Expected against actual

A supplier describes a lot in a spreadsheet before it arrives. A lot therefore exists, with expected quantities and a supplier, before a single device is touched — and receiving becomes **reconciliation** rather than data entry.

The manifest importer parses CSV and XLSX **in the browser**, not on the server. The files are small and the formats are inconsistent, and the operator needs to see what the system understood before any of it is committed.

The reconciliation has a rule worth stating, because it is an instance of the paper's thesis applied to a business process: **a false "extra" is an accusation**. If the system reports that a device arrived which was not on the manifest, it is implicitly saying somebody added it. That claim has to be correct, so the matching is conservative and unmatched items are presented as unmatched rather than as discrepancies.

4.3 Stock as a derived balance

Consumables are modelled as StockLine plus StockMovement. The quantity on hand is **derived from the movements**, never a number a person edits. Every change carries a reason. The result is that "we have eleven caddies" is a statement with a history behind it rather than an assertion somebody typed, and the low-stock alert that replaced a removed feature (§13) answers a question the warehouse actually asks.

The pallet model has an equivalent invariant, arrived at after two rebuilds: **a merge moves lines, it never copies them**. Merging pallet B into pallet A must leave the total count unchanged. Copying would double stock; moving cannot. Stated that way the rule is obvious, which is exactly why it needs to be written down — the implementation that copies looks correct in every test that does not add up the totals afterwards.

4.4 Ownership, and the lesson that generalises

With several managers buying and processing stock, the system had to answer *whose stock is this, and who did this?* Ownership was added to batches and then enforced in six passes:

Pass	Scope
E1	Batch reads
E2	Asset reads, via their batch's owner
E3	Dashboard, reports and certificates
E4	Write guards, and admin reassignment
E5	The PowerSync upload path <i>and</i> the sync rules
E6	The activity log

E5 is the one worth extracting. Scoping the HTTP API is not sufficient when clients synchronise a local database directly. Without scoping the sync *rules* as well, a manager's phone would download rows it could not have fetched over HTTP — the guard on the front door held while the side door stayed open. Any system combining a permissions layer with a sync engine has two separate enforcement surfaces, and securing one reads exactly like securing both.

Two scoping decisions were made explicitly and recorded rather than discovered later: **scoping applies to managers only** (technicians are floor staff who scan into any lot, and scoping them breaks the job), and **technician-created lots belong to an unowned pool** visible to all managers, without which a technician's work would have become invisible to everyone.

4.5 Append-only history

The activity log is system-wide and append-only, and asset history is kept rather than overwritten. This is the data-model expression of the invariant in §3.3: *evidence flows inward, and the record is never edited to agree with it*. When a second audit contradicts the first, both survive, and the question "what did we believe, and when?" stays answerable.

It also constrains a feature the business asked for later. Deleting a user would destroy the audit trail that names them, so users are **disabled, never deleted** — a rule that then required its own work on session invalidation, because a disabled account with a valid token is still signed in (§10.3).

5. Layer 2: offline-first capture

R1 says capture must work with no connection. That single requirement produces the component with the worst failure mode in the entire system, because **a queue that loses work looks exactly like a queue with nothing to do**.

5.1 The architecture

The client holds a local SQLite database. PowerSync streams rows down according to sync rules and carries writes back up. The technician's phone is therefore a full participant rather than a thin client: scanning works in a loading bay with no signal, and reconciles when signal returns.

The web application, by contrast, was **deliberately not made offline-capable**. Only the scanning and capture paths are offline-first; reporting and administration assume a connection. Offline reporting would have meant replicating business rules into the client for no operational gain.

5.2 Four faults in the first week, three of them silent

PowerSync did not work out of the box, and each failure is load-bearing enough that it is still shaping the codebase:

Fault	How it presented
Worker assets never copied into the build	Sync hung silently — indistinguishable from nothing to sync
JWTs signed with the wrong kid	Auth failed for a reason documented nowhere obvious
Multi-word columns dropped on upload	Data looked saved on the device and arrived incomplete
One deleted batch reference	Wedged the entire upload queue behind it, indefinitely

Three of the four announced nothing. The fourth announced nothing useful: an unresolvable foreign key stopped every queued write behind it, forever, and the device carried on accepting work.

That last one produced a rule that recurs throughout the project and is restated in §17: **a record the server refuses must become visible, not be retried forever in silence**. A queue that retries indefinitely is not resilient; it is a way of hiding a permanent failure behind a temporary-looking one.

5.3 Scanning: the decision, and what a warehouse did to it

Every device enters the system through its serial or service tag. Typing them is slow and wrong; the labels are small, worn, and often carry no barcode at all.

A commercial scanning SDK was evaluated and **rejected in favour of free platform capabilities**: the browser's native `BarcodeDetector`, plus full-resolution camera OCR for text-only labels. The reasoning was that a paid SDK is a per-device licence on hardware the business keeps buying, for a job the platform can already do. The accepted trade-off is that `BarcodeDetector` support varies by browser and OCR on a worn label is worse than a dedicated engine's.

The first version worked in an office and badly in a warehouse. Three corrections, each generalisable:

Know what you are looking for. A Dell label carries the service tag, the express service code, FCC identifiers, regulatory model numbers and certification marks. Naive OCR returns whichever string it read most confidently — frequently a regulatory number. The fix was to encode what a vendor's tag actually looks like, rather than trusting confidence.

A still photograph beats a video stream. An aiming box was added to help the operator, then removed. Cropping discarded the pixels OCR needed, and a live video frame is lower resolution than a still. Full-frame detection plus a full-resolution still photo outperformed the thing that looked more helpful.

Show the operator what the software is doing. The torch toggle failed on some devices and nothing indicated which detection engine was running, so a failure to scan was indistinguishable from a failure to aim. On-screen engine status fixed more perceived bugs than any detection change.

One further correction is a data-model point: the same device scanned from a barcode and read by OCR produced **different strings**. A single normaliser, applied to both paths, is what makes the two agree.

5.4 The queue that had to survive a power cut

The audit station kept queued records **in memory**. A technician who audited six machines with no network and then powered the bench down lost all six, and nothing said so. On a bench where machines are powered off as a matter of course, that was a question of when rather than whether.

The queue moved onto the USB stick — the one component guaranteed to still exist after a power cut.

A month later the same component produced the most serious defect in the whole catalogue. A queue file that existed but **could not be fully parsed** was treated as empty, and then rewritten — destroying the records it had

failed to read. It is the only instance in §12's dataset that reached nobody, because it produced no wrong sentence: it produced no sentence at all.

Making storage durable and making it safe to read are two different jobs. The first was done deliberately and the second was assumed.

5.5 Two instrumentation faults with the same shape

Ping is not the internet. The station's connectivity check tested one address by ICMP. A site that filters ping — as this one does — produced a red failure on a station that was uploading perfectly well. The check now tries several addresses over both protocols, shows the routing table, and, decisively, judges the station by **whether it can reach the ALS API**, not by whether it can reach the internet in the abstract.

A silent success looks like a hang. An OS restore writes several gigabytes with no output. The operator sees a still screen and cannot distinguish a working restore from a dead one. The system knew it was fine and was not saying so.

Both are the same defect as the queue, pointed the other way: the first reported a failure that had not happened, the second failed to report a success that had. In each case the code held the information and discarded it before it reached a person.

6. Layer 3: the audit station

The audit station is a bootable USB stick. A technician plugs it into a machine that arrived that morning, boots it, and gets one full-screen interface that examines the machine, erases it, and uploads what it found.

Booting the customer's own Windows was never an option. Many arrive locked or encrypted; booting changes the machine, which is evidence (R3); and a great deal of what needs to be read — the registry, the hidden partitions, the raw drive — is easier and safer to reach when Windows is not running. **The machine's operating system is never started.**

6.1 The base, and the constraint that chose it

The station runs Ubuntu 24.04. That choice was made for **Secure Boot**: a significant share of incoming machines will not boot an unsigned image without a firmware change the technician should not be making on a customer's hardware.

The live session, though, is amnesiac. Every `apt install` is discarded at reboot — and without `nvme-cli` an NVMe drive cannot be secure-erased, which is most of what comes through the door. So the station carries a **casper overlay layer**: an additional squashfs stacked on the stock image, holding the extra packages and the kiosk.

How casper selects layers had to be read **out of the stick's own initrd**, because most of what is written about it online is wrong for 24.04. Selection is driven by one variable:

```
LAYERFS_PATH=minimal.standard.live.squashfs
```

casper builds the stack by repeatedly stripping the last dot-component, and the **longest name ends up highest priority**. A layer named `minimal.standard.live.als.squashfs` therefore extends the chain and sits on top.

The trap, which the build script now refuses to be talked out of: the chain only walks *up*, by stripping. Name the layer `als.squashfs` and the chain is just "als" — casper mounts a few hundred kilobytes **as the entire root**, finds no `/sbin/init`, and panics. **The name must extend a chain whose every ancestor already exists.**

6.2 Seven attempts to make a browser fill a screen

The station is an appliance: one interface, full screen, no desktop, no browser chrome, no way to end up somewhere else. The hardware it runs on is whatever came through the door — any manufacturer, any graphics chip, any panel, sometimes no working display driver. There is no fixed target to develop against.

Approach	Why it failed
Firefox kiosk mode	Hides the toolbar, opens at ~90%, never fills
Cage (Wayland kiosk compositor)	Correct when present — not on the image
Auto-install Cage at boot	Needs a network; the bench often has none
X fallback with a window manager	Another dependency not on the image
Window resize via an external tool	Deterministic, still an install
python + libX11 via ctypes	Worked. No packages, any display

The lesson, paid for over seven attempts and fifteen commits:

On a live-boot appliance, a dependency you have to install is a dependency you do not have.

Every approach needing a package worked on the development machine and failed on the bench. What shipped talks to libX11 through ctypes and resizes whatever top-level window appears, using only what the stock image already carries.

Progress became possible only after a **Display readout** was added to the status bar showing the resolution *and which of the five launch paths had actually run*. Until then every failure looked identical: a window that was not full screen. Most of the subsequent progress is attributable to being able to see which mechanism had fired — the same instrumentation principle as §5.5.

6.3 Who owns a keypress

An appliance has to survive its operator. During functional keyboard testing a technician pressed keys on a Lenovo laptop and the entire interface disappeared to a black screen. Investigating produced a model that turned out to be generally useful:

Layer	Owns	Can software refuse it?
The page	Ordinary keys, Escape, function keys	Yes — the event can be cancelled
The X server	Virtual-terminal switching	No — intercepted below the client
The machine	Brightness, display output, vendor app-commands	No — firmware or the keyboard controller acts first

Only the first layer offers a veto. The second required disabling server key bindings at the X level; the third required clearing the offending keysyms outright, because the machine acts before any software is consulted. A test that asks a technician to press every key must therefore be defended at all three layers — and the parts that cannot be defended have to be designed around rather than trapped.

6.4 The station's hardware is not the machine's

The very first version of the audit tool enumerated block devices and took the first one as the machine's drive. The first device was **the USB stick it had booted from**. Every audit recorded the stick's model and capacity as the machine's storage.

That defect is the ancestor of a rule now enforced in several independent places: never report the station's Ubuntu as the machine's operating system; never wipe the boot device whatever it reports itself as; skip removable and USB devices in the storage scan. R4 exists because this confusion is easy to write and invisible in a result that otherwise looks complete.

6.5 Accountability at the bench

Certificates are legal documents, so the station requires an **operator sign-in**: the person who performed a wipe is recorded against it, not merely the station that ran it. The station authenticates with a restricted account, never an administrator (R6), and the kiosk service was hardened so the interface cannot be escaped into a shell.

6.6 How code reaches a warehouse

There is no deployment pipeline to a USB stick on a bench. Getting a change onto the stations is a physical act, and its cost differs by an order of magnitude depending on which file changed:

Change	What it takes
Interface, probes, autostart script	Stick sync. Copy files, checksum each, done
Anything inside the overlay layer	Layer rebuild plus a reboot on the audit machine

This boundary is a real design input, and one decision shows why. The Autopilot work needed an event-log reader that is not on the stock image, so it was added to the layer build — gating the whole feature behind a rebuild. The rebuild could not be run off the target machine, and the reason is more interesting than the absence of build tools on Windows: the build has a safety gate that simulates installing the packages against the **host's** package database, to catch a package that would be an *upgrade* of something already on the live image. In a container, that database is the container's. The gate would report "none of these is an upgrade" not because it is true but because it **cannot see** — passing vacuously, silently disabling the check that exists to prevent exactly that failure.

The alternative shipped instead: the live session is writable and the stick is already synced, so the packages ride on the stick and are installed at boot. They stay in the layer list as well; whichever arrives first wins, because the installer returns early if the tool is already present.

Two further details of the sync path are worth recording, because both hid real faults. The stick carries a **version stamp**, without which a bug report cannot be tied to a build. And the sync **flushes the write cache**, because Windows reports a copy to removable media as complete before the data has left the buffer — pull the stick immediately and you get a truncated file that verifies as *present*.

7. Layer 4: deep inspection

Reading a machine's hardware is the easy half. The commercially and legally interesting questions are about its *state*: is it locked to somebody else's organisation, is it encrypted, is there a BIOS password, how much life is left in the drive, what operating system is installed. All of these must be answered about a machine whose operating system is never started.

7.1 The constraint, stated as a capability

The station boots Linux, mounts the Windows volume **read-only**, and parses its registry hives offline. Everything follows from that one sentence.

Impossible	Available instead
Live management-status queries	The registry keys those tools read

Impossible	Available instead
PowerShell, WMI, any live API	The hive files, parsed directly
Asking the machine's OS anything	The artefacts its OS left behind

This is not a limitation to engineer around; it is the shape of the problem. And it has one genuine advantage over the live equivalent: parsing a hive file directly **ignores the access-control list** that would block the same query on a running system, so the SECURITY hive — normally unreadable even to an administrator without extra work — is simply a file.

7.2 What a powered-off Windows will tell you

Four management states matter to a resale business, because each one can make a machine unsellable or unusable to its buyer:

State	Where it is read from
Entra ID (Azure AD) join	The cloud-domain-join key: tenant identifier, tenant display name, device identifier
Active Directory domain join	The SECURITY hive: the primary domain SID, the domain names, corroborated by the machine account and cached logons
MDM enrolment	The enrolment keys, keeping the organisation's domain only
Licence and activation	The firmware-embedded licence marker, recorded as <i>present</i> , never transcribed

Two rules govern all four, and both are about restraint rather than capability.

Personal identity is never read. The cloud-domain-join key contains a user's email address. It is a former employee's identity, it has no bearing on whether the machine can be resold, and it is never read. Only the **organisation's domain** is recorded. The same precedent applies to MDM enrolment, where only the domain part of the enrolling account is kept.

A product key is recorded as present, never transcribed. The system needs to say a licence exists. It does not need to hold the key, and holding it would make the audit record a thing worth stealing.

7.3 Three mechanical traps, and the one that got through

Reading hives offline is fiddly in ways that are worth recording, because each of these cost a debugging session and none is documented anywhere convenient:

1. **The values live in the key's default, unnamed value**, which the convenient command-line getter cannot fetch at all — it needs the interactive shell's value-listing command.
2. **They are binary UTF-16LE**, and the shell drops NUL bytes silently, so the string arrives mangled unless the NULs are stripped deliberately.
3. **A binary value is printed in a hex:04,00, . . . form. The name extractor took the first token and reported the machine's domain as the literal string "hex".**

The third one shipped, and appeared on a real audit. It is worth dwelling on because it runs *opposite* to everything else in this paper: it is a false **presence** — a machine asserted to be domain-joined to an organisation called "hex" when it was not domain-joined at all. It was noticed within a day, precisely because a false alarm is the kind of error somebody chases. §12 returns to it as the exception that demonstrates the direction of the other 32 defects is structural rather than selective.

7.4 Locks, and the promotion rule

The station runs a set of independent detectors — encryption, BIOS password, firmware lock, management enrolment, hidden partitions — and each returns one of three values, never two: **present**, **absent**, or **could not be established**.

The device-level verdict is then computed by a rule that is the operational heart of the thesis:

Any detector returning "could not be established" promotes the whole device to UNVERIFIED.

Not *unlocked with a caveat*. Not *probably fine*. A device on which one of five checks could not run is a device whose lock status is unknown, and it says so. This is deliberately conservative and deliberately expensive: it produces more machines needing a second look, which is the correct trade when the alternative is selling somebody a laptop that turns out to belong to a bank.

The rule exists in code rather than in a convention, and it is enforced at the point where the detectors are collected: a detector that returns success while having filed no finding at all is itself recorded as an unknown, rather than being treated as a silent pass.

7.5 Drive health: never "Unknown"

A buyer's first question about a second-hand machine is *how much life is left in the drive*. SMART answers it, but not in a form anyone can act on: raw vendor-specific counters, with different fields on SATA, NVMe, eMMC and SAS.

The requirement, set by the business, was uncompromising:

Drive health is a percentage, in named bands, and it is never "Unknown".

Where health genuinely cannot be measured, the row states **what could not be measured and what to do about it** — for example, that the drive sits behind a RAID controller, together with the instruction to switch the storage mode in firmware and rescan. That is an answer. "Unknown" is not; it is a blank pretending to be one.

Three implementation decisions made it hold:

- **One formula, in one place**, with a presentation layer per application, so a drive reading 74% on the bench is not "Caution" in the kiosk and "Fair" in a report. Identical wording across three surfaces was treated as a requirement, not a nicety.
- **Show what was measured, not a fresh guess**. The kiosk was re-probing drives without privilege and getting *worse* answers than the capture already held. Displaying the captured measurement fixed it.
- **Do not grade a drive down for a warm afternoon or a bad cable**. Temperature and interface CRC errors were dragging grades down; neither is a wear indicator.

7.6 Reading the installed operating system

The station reports which Windows is installed by reading it from the machine's own registry — never by inferring it, and never by reporting the station's own Ubuntu.

One case governs the design. A **BitLocker-encrypted machine** presents a volume that cannot be read. The naive implementation looks for Windows, fails to find it, and reports *no operating system installed* — which is both wrong and commercially significant, since it makes a working encrypted laptop look like a bare-metal box. An encrypted volume must read as *encrypted, therefore not readable*, which is a different sentence from *empty*.

This is the same defect shape as everything else in this section, which is why the layer that produces the most valuable findings also produced the majority of the defects catalogued in §12.

8. Layer 5: erasure and certification

This is the layer the business exists to produce. Everything else in the system is, in the end, apparatus for making one document true: a certificate saying that the data on a specific drive in a specific machine is gone.

8.1 A ladder, not a technique

Drives differ, so the engine holds a ladder of methods and descends it:

Method	Applies to
Cryptographic erase	Self-encrypting drives — discard the key
Firmware secure erase / sanitize	SATA and NVMe, at the controller
Block erase	NVMe
Overwrite	The fallback when nothing above is available or accepted

A method may be refused by the drive. When that happens the operator is told **which method actually ran and why the faster one was declined**, and that statement reaches the certificate. This is the earliest appearance of the rule that now governs the whole document:

The certificate says what was actually done, not what was asked for.

Every erase is followed by a **read-back**: the drive is read to confirm the data is gone, rather than trusting the command's exit status.

8.2 Seven things that were wrong, and one of them was already on paper

The engine shipped in July was structurally sound and made claims it could not support. A remediation programme in September — a 42-step plan across four tracks, roughly 100 commits, the largest single body of work in the project — fixed seven distinct failures. They are worth listing individually, because each is a different way for a true-looking document to be false:

1. TRIM was being certified as an erasure. Issuing a TRIM marks blocks unused; the data may remain readable. This one had already produced wrong certificates, so the fix was not only to stop doing it but to **de-certify records already issued** from earlier sticks. A defect that has reached a customer is not fixed by preventing the next one.

2. An unreadable drive "verified". The read-back pass treated a drive it could not read as having passed — the exact shape of §12's defect class, in the most consequential place it could occur. Read-back now happens after every erase, and "controller-confirmed" was dropped as a category entirely: the controller reporting success is not evidence that it succeeded.

3. The verdict was per machine, not per drive. A machine with two drives, one of which failed to wipe, was being called wiped. Each drive is now decided on its own evidence, the machine's status follows from the per-drive verdicts, and a certificate is **refused** when a sibling drive's wipe failed.

4. Drives were identified by path or serial alone. Paths move between boots; serials are sometimes blank or duplicated. The engine now identifies a drive by the combination of what it reports about itself, records each drive's own identity and timings, and **refuses to act on the wrong drive**. The failure mode being prevented is erasing a disk that was not the subject.

5. NVMe namespaces were mishandled. An NVMe drive can present several namespaces. The engine first refused the second one, then learned to wipe every namespace in turn, sanitize the correct controller first, and never issue a partial format.

6. Hidden areas were never checked. SATA drives can carry host-protected or device-configuration areas — regions hidden from the operating system that an erase does not reach. These are now checked before wiping, by reading the kernel's view of the drive's size, and **without making any permanent configuration change to the customer's hardware.**

7. The sanitize result was parsed as English. The status was matched against text strings that vary between tool versions. It is now read as numbers.

The structure built in July — ladder, fallback reason, read-back, certificate — was sound enough that all of this was a hardening rather than a rewrite. That is worth recording in a paper about getting things wrong: the original design was right and its *claims* were overconfident, and those are separable failures.

8.3 Making the document tamper-evident

An erasure certificate is a PDF, and a PDF can be edited. Nothing stopped somebody altering a serial number, a date, or a verdict on a document the business had issued, and nothing let a buyer confirm that the copy in their hand was the one that was issued.

Certificates are therefore **stored, signed and chained**: each entry is linked to the one before it, so altering any entry breaks the chain from that point forward. The ledger is tamper-evident rather than merely tamper-resistant — it does not prevent an edit, it makes an edit detectable.

A buyer can check a certificate through a **public endpoint reached by a QR code on the document**, which reports whether the certificate is genuine and what it says. It is **off by default**: a public endpoint on a system holding customer data should be an explicit decision, not something a deployment inherits.

8.4 What making it public cost

Three separate pieces of work exist only because that endpoint is unauthenticated, and they generalise to any public verification surface:

- **Bound the cost of a single check.** An unauthenticated endpoint that does real work is a denial-of-service surface.
- **Spend the budget only on certificates that exist.** A request for a non-existent certificate must be cheap, or the budget is consumed by enumeration attempts.
- **Never render a link that claims more than it can deliver.** The internal asset page could hang while checking the ledger and — worse — could display a verification link for a certificate that was not actually verifiable. A link labelled *verified* that leads nowhere is a false claim in exactly the sense this paper is about.

8.5 Keys

The signing key never leaves the server. Verification uses the public key; the private key has never appeared in a log, an error message, or a conversation about the system, and the discipline of not exposing it was treated as a standing rule rather than a one-off precaution.

Keys are rotated, and a certificate signed with a retired key must still verify. When a retired key cannot be read, the failure **names the entry at fault** rather than failing the whole ledger — otherwise one unreadable key turns every historical certificate into an unverifiable one.

8.6 What the certificate refuses to say

Three refusals define the document as much as its contents:

It will not say	Because
That an unreadable drive was verified	Failing to read is not a pass (\$12)

It will not say	Because
That a machine is wiped when one of its drives is not	The verdict is per drive (§8.2)
That the fastest method ran when it was refused	The document reports what happened (§8.1)

Two questions remain open and belong to the business rather than to engineering: whether a machine whose drive is hidden behind a RAID controller should be certifiable at all, and whether a lock discovered *after* issue should reach a certificate already in a buyer's hands. Both are recorded in §16 rather than silently defaulted.

9. Layer 6: functional testing, and the human as an instrument

Every layer so far describes what a machine *is*. None of them establishes whether it **works**. A laptop with a dead speaker, a stuck key, a broken camera or an unresponsive trackpad reads as perfect on a specification sheet, and is then sold as perfect.

9.1 The principle

The station's interface is a web page. A web page cannot measure whether a speaker produced sound, or whether a camera image is in focus. The governing rule for this layer was therefore set as a prohibition:

Do not pretend the browser can diagnose hardware it cannot measure.

What the page *can* do is drive the hardware and **ask a technician what happened**. Every result in this module is a human's answer, and is recorded as one — the stored finding carries the fact that it was confirmed by a technician, not measured by software.

This is an unusual thing to build deliberately. It is also the honest design: the alternative is a probe that emits a confident verdict about something it did not observe, which is the defect this entire paper is about, dressed as a feature.

9.2 Could-not-run is not failed

The companion rule matters as much:

A test that could not run is ATTENTION, never FAILED.

Marking a component failed because the station could not test it would condemn working hardware — a machine downgraded, or scrapped, because a driver was missing on the live image. The three-valued vocabulary from §7.4 applies here with the polarity reversed: in lock detection, an unestablished result must not read as *safe*; in functional testing, an unestablished result must not read as *broken*.

Both rules are the same rule. **The third value has to exist, and it must not collapse into whichever of the other two is convenient.**

9.3 The seven tests

Test	What the station does	What the technician answers
Speaker	Unmutes, plays a tone on each side	Did you hear it, both sides?
Keyboard	On-screen layout, lights each key as pressed	Good, or has an issue
Camera	Live preview	Is the image good?
Screen	Full-screen colour fields	How many dead pixels?

Test	What the station does	What the technician answers
Trackpad	Tracks movement, buttons and scroll	Good, or has an issue
Microphone	Records and plays back	Could you hear yourself?
USB ports	Counts the ports that responded	—

The USB row is the one without a human answer, and is consequently the one that produced a false absence: zero ports responding was reported as *zero working ports* rather than as a test that did not run. It appears in §12's catalogue, found on real hardware by a technician who knew the laptop's ports worked.

9.4 The keyboard test, which took five attempts

The keyboard test is the most instructive thing in this layer, because every attempt failed for a different reason and none of the reasons were about keyboards:

1. **A pressed key activated the focused control.** Space and Enter fired the verdict buttons, so testing the keyboard answered the question the test was asking.
2. **Browser shortcuts fired.** Detection moved earlier in the event sequence so the page could claim the key before anything else acted on it.
3. **Function keys triggered browser features.** On one machine a function key opened a bookmark panel over the interface.
4. **A virtual-terminal switch blanked the screen**, which no page-level code can prevent (§6.3).
5. **Machine-level keys acted before any software saw them** — brightness and display-output keys on a Lenovo laptop blacked out the interface entirely.

The first three were fixable in the page. The fourth needed the X server reconfigured. The fifth could only be addressed by removing the offending keysyms altogether, because the firmware acts first.

The sequence is a small case study in a general problem: **a test that asks a human to exercise arbitrary hardware is a test that invites the hardware to interfere with the test.** The fix is not cleverness in the page; it is knowing which layer owns each input and defending at the right one.

9.5 Why this layer ships by file copy

The entire module reaches the stations by stick sync, with no layer rebuild (§6.6), because it is interface and probe code rather than packages. That was a deliberate constraint on the design, not a happy accident: a functional-test module that required rebuilding a filesystem image would have been revised perhaps twice instead of the five times the keyboard test actually needed.

10. Layer 7: the back office

The web application is where the evidence produced at the bench becomes a business: receiving and reconciliation, the inventory hierarchy, pallets, sales status, reporting, certificates, and user administration. Thirty pages.

It is the least novel layer in the system and it produced two findings worth reporting anyway — one about where authorisation actually lives, and one about a page that contradicted itself.

10.1 Permissions, inverted

Permissions had grown case by case since the ownership work in July. Each endpoint decided for itself, which meant **the default was open**: a route added without a guard was reachable by anyone signed in, and nobody could state what a given role could do without reading every controller.

The rebuild inverted the default. **A route with no declared permission is refused.** Forgetting to guard a new endpoint now produces a visible refusal in testing rather than an invisible hole in production. The failure mode moved from silent to loud, which is the same move the rest of this paper keeps making.

Two consequences were made explicit rather than left to convention:

The UI hides what the API refuses, and the API is what enforces. Hiding a button is a courtesy, never a boundary.

Every gated control in the application is gated on **the same permission the API checks**, and navigation is built from what the signed-in user may actually do — including landing rules that send each person to a page they can use, and deep links that are honoured rather than bounced.

The second consequence was overdue. Every audit station's USB stick had been authenticating **as an administrator**. A lost stick was a full compromise of the system. Stations now hold a least-privilege account that can file audits and nothing else (R6).

10.2 Deleting a user is not an option

The audit trail names the people in it, so deleting a user would destroy the record of who did what. Users are therefore **disabled, never deleted** (§4.5).

That decision creates an obligation that is easy to miss: a disabled account holding a valid token is **still signed in**. Bearer tokens are not revocable by themselves — the server has already said yes, and nothing about disabling a row reaches the token in somebody's browser. Making "disabled" mean anything required an explicit revocation path and the client handling rejection properly, so that a disabled account is ended rather than merely marked.

This is the authorisation equivalent of the paper's thesis: a state change that is recorded but not enforced is a claim the system cannot support.

10.3 A dashboard that disagreed with itself

The operational dashboard is computed as **one SQL roll-up** rather than many queries assembled in JavaScript. The performance gain was secondary; the real reason is consistency. Figures computed in separate queries at slightly different moments disagree with each other, and *a dashboard whose numbers contradict each other is worse than no dashboard*.

The defect worth recording is subtler than a wrong number. The headline "In stock" figure **totalled five statuses**; clicking it navigated to a filtered list asking for **one**. Neither the number nor the list was wrong on its own. The defect was that two parts of one page answered different questions while appearing to answer the same one — and it only surfaces when somebody actually clicks.

The same shape had already appeared once, in a reports filter. It is worth naming as its own small class: **a summary and its drill-down must be computed from the same predicate, or the page lies at the moment a user trusts it most**.

10.4 Reporting

The reporting section was rebuilt from a set of tables into eleven analytical slices — sales, batches, pallets, warehouse, users, consumables, suppliers, activity — with filters and spreadsheet and PDF export. It is the least architecturally interesting part of the system and among the most used, which is a common and under-reported combination.

One rule carried over from the rest of the system: reports respect ownership scoping (§4.4, E3), and a printed report **says who generated it**. A document that leaves the building carries its provenance.

10.5 Consistency as a defect detector

Two pieces of cross-cutting work belong here because of what they *found* rather than what they changed.

A **visual system pass** brought thirty-one pages into one design language. In doing so it exposed ten functional defects. Pages that had been written independently had diverged in ways nobody noticed until they were placed side by side — controls that did nothing, states that were unreachable, labels that meant different things on different screens.

A **WCAG 2.2 AA accessibility pass** did something similar. Its most instructive moment is a correction: a lighter field border was introduced for visual reasons, **failed contrast**, and had to be adjusted — the trade-off being written down rather than quietly reverted.

Both passes are the same technique: forcing a system to be consistent is a way of discovering where it is wrong. Inconsistency is cheap to tolerate one page at a time and expensive in aggregate, and the aggregate is invisible until something makes you look at all of it at once.

11. The knowability framework

Building the inspection layer forced a question that turned out to be more general than the system: **what can a computer that is not running be made to tell you about itself, and what can it not?**

The answer is not a single line between possible and impossible. It is four levels, and the distinction between them is operational — it determines whether a claim can be measured, inferred, recovered only at a moment that has already passed, or not obtained at all.

11.1 Four tiers

Tier	The claim is...	Example from this system	Costs
K1	Measured. The hardware reports it on request	Processor, memory, SMART attributes, TPM presence	A probe
K2	Recovered. The absent OS left an artefact behind	Domain join, encryption state, installed Windows, event logs	A parser, and knowing where to look
K3	Perishable. Obtainable only before an earlier step destroyed it	The device hardware hash, available before the wipe	Process change, not code
K4	Withheld. The answer lives with a third party, behind their authentication	Autopilot registration	Not obtainable. Full stop

K1 and K2 are engineering. K3 is a **process** problem wearing an engineering costume — no amount of code recovers something that has been overwritten, and the fix is always to capture it earlier. K4 is neither: it is a boundary, and the only correct engineering response is to report the question as unanswered.

The practical value of the framework is that it tells you **which kind of effort will work**. Time spent writing a cleverer probe for a K3 fact is wasted; time spent on a K4 fact is worse than wasted, because any answer it produces is a guess wearing the costume of a measurement.

11.2 The worked example

Microsoft Autopilot is a provisioning service. A device registered to an organisation will, at its first boot after a reset, contact Microsoft, discover it is claimed, and enrol itself into that organisation — potentially in somebody else's hands.

For a refurbisher this is the worst commercial failure available: a machine is bought, audited, wiped, refurbished, sold, and at the buyer's first boot it is claimed by an organisation nobody knew about. The money is already

spent.

So the question *is this machine Autopilot-registered?* is the single most valuable thing the system could answer. It is also K4.

The registration is a row in Microsoft's cloud, keyed to the device's hardware hash. It is not on the disk. Wiping does not touch it. Reinstalling Windows does not touch it. The device learns about it only by asking, at first boot, over a network — a sequence that ends in an authenticated request for the device's provisioning profile, returning either a profile or a not-registered error.

There is no public interface into that final step for anyone except the tenant that owns the device. The engineer who built Autopilot at Microsoft has documented the flow publicly and described the ability to check a device's status as something that **"would be nice to be able to check"** — that is, as a thing that would be useful rather than a thing that exists.

When the person best placed to know describes a capability as desirable rather than available, that is as close to a definitive answer as this kind of question gets.

11.3 What was built instead

Rather than guess, the system was given three of the four tiers and told to be explicit about which one produced each answer:

	What it does	Answers a wiped disk?
Tier 1 — offline artefacts	Reads the Autopilot traces still present on the mounted volume: the enrolment profile, the diagnostic entries, the event log the OS wrote	No. Only if the disk still holds them
Tier 2 — capture at intake	Record the hardware hash <i>before</i> wiping, so the machine can be checked against a tenant later	No — it makes the later check possible
Tier 3 — observe first boot	Record what the machine's own first-boot setup screen actually said	Yes, definitively
Tier 4 — reimplement the query	Perform the device's authenticated request independently	Yes, until Microsoft changes it

Tier 3 is the only one that yields certainty, and it does so by abandoning the attempt to deduce the answer and instead observing the event where the machine itself finds out. The machine will tell you it is claimed at the moment it discovers it, and a technician can record what appeared on screen. It is not elegant. It is evidence.

Tier 2 is blocked, and the reason is instructive. The hardware hash is the only identifier Microsoft will act on, and the station currently wipes it away without recording it. It cannot be read from Linux: it comes from a Windows management interface that requires Windows to be running. The established route without the installed OS is a pre-installation environment, which means adding a Windows PE image to the stick, the vendor toolkit, and a real machine to prove it on. **It is blocked rather than skipped**, because writing it blind would produce something nobody could verify — and an unverified hash capture is worse than none, since it would be trusted.

One honesty point belongs in the record if Tier 2 is ever built: a hash captured in a pre-installation environment is **not byte-identical** to one captured from the full operating system, because several components are missing. It is close enough for the tooling people use daily, and the record must say which kind it holds.

Tier 4 was recommended against, and the recommendation is a judgement rather than a technical impossibility. Reimplementing an undocumented authentication sequence would probably work, for a while. But the output of this system is a legal document, and a certificate whose most valuable claim rests on an undocumented protocol that a third party may change without notice is a liability dressed as a feature. The failure mode is not that it stops working — it is that it keeps *appearing* to work while returning something that no longer

means what it used to.

11.4 The general form

Strip out the vendor and the framework applies to any system that reports on state it does not own:

1. **Classify every claim by tier before implementing the probe.** The tier determines whether code, process change, or nothing will produce the answer.
2. **A K3 fact needs a process change, not a better probe.** If a fact is destroyed by a step in your own pipeline, the fix is upstream of that step.
3. **A K4 fact must be reported as unanswered.** Not "no evidence found", which a reader will take as "not registered" — but an explicit statement that this question has no offline answer, with the reason.
4. **The tier must reach the record.** It is not enough to know internally that a finding was inferred rather than measured; the person reading the document has to be able to tell.

Point 3 is where this framework and §12's defect class meet. A K4 question answered with the *absence* of evidence is a false absence with extra steps — and it is the most dangerous kind, because the probe genuinely ran, genuinely found nothing, and genuinely could never have found anything.

12. Evidence integrity: the false absence

Every layer in Part II has, at some point, reported something it had not established. This section names that defect, measures it across the whole project, and reports an uncomfortable result about what naming it achieved.

12.1 The class

A **false absence** is a failed observation reported as a confirmed negative. It requires four conditions, all of which must hold:

1. A probe reads state the program does not control — a device, a filesystem, a registry hive, a remote service.
2. The probe can fail **without raising**. It returns a value.
3. At the point of use, that failure value is **indistinguishable from a genuine negative**: empty string, empty list, zero, false, missing key.
4. The value is **reported as a finding**, unqualified, to a person or onto a record.

Condition 3 is the operational test, and a developer can apply it at the keyboard: **does this probe's failure value equal its negative value?** If it does, any assertion built on it is unearned.

The class has a direction, and the direction is not incidental. Failure produces empty, zero, false, absent — and in a system that examines things for problems, those are exactly the values that mean *no problem*. A probe that fails resolves, every time, towards the reassuring answer.

The converse defect is self-correcting. A probe whose failure produced a spurious *lock* would be chased down within a day, because a false alarm costs somebody an hour and they complain. This is not hypothetical: §7.3 describes one, a binary registry value that made machines appear domain-joined to an organisation called "hex". It was caught almost immediately. A false absence costs nothing visible, and gets certified.

12.2 What was catalogued

32 call sites across 15 fix commits, over 82 days and 526 commits. The dataset, with the inclusion criteria written before the analysis, is published in `paper/evidence/`.

Finding	
Failed towards the reassuring answer	32 / 32
Reached a person, on screen or in a document	31 / 32
Could affect the erasure certificate	18 / 32
In the audit station rather than the server or web app	28 / 32

The reported sentences were: *no lock detected, no operating system installed, no TPM detected, not enrolled, not joined, no hidden drives, no encrypted volumes, no BIOS password, limitations: none reported, 0 ports responded, not present, movement and buttons work*. Every one asserts that something which would have been a problem is absent.

The single instance that reached nobody is the one that destroyed data: the unreadable queue file of \$5.4, treated as empty and then rewritten. It produced no wrong sentence because it produced no sentence at all.

Twenty-three of the 32 are in two files, and 28 are in the station — the component that boots a customer's machine and examines it. **The class concentrates where code touches state it does not control**, which is the sharpest practical guidance this catalogue offers.

12.3 Why the tests did not catch them

Eighteen of the 32 sat on code paths that **already had tests**. Those tests caught **none** of them. Examining each fixture gives two categories, which are the same blindness from opposite ends:

	Count	The fixture...
Working probe supplied	14	...gave the probe something it could read
Adverse state never constructed	4	...never built the subject that matters

Neither is negligence. A fixture is written from the author's mental model of the path, and that model is of the path *working*. Catching this class needs a fixture in which **the probe fails and the subject is dirty** — two independently unlikely conditions, jointly unrepresented.

During this work, four existing tests broke when a failure check was added. In every case **the fixture was wrong, not the check**: each stubbed a command that could not fail. One test's own name asserted the defect.

12.4 The recurrence finding

On 2 September a commit landed titled **"Never let a failed probe read as a negative."** It repaired six lock detectors, **named** the class in its own title, **documented** the reasoning in the affected functions, and **defended** the fix with 75 new lines of tests — a third of the diff. By any ordinary standard the lesson had been learned and recorded.

Nineteen days later, a systematic search of the same codebase found twenty-six further instances. That figure conflates two claims of unequal strength, so they are separated:

	Instances	What it means
Fixed by the naming commit	6	The instances that prompted the naming
Introduced before it, found later	18	Naming failed to find these
Introduced after it	8	Naming failed to prevent these

The eighteen are a failure of *search*, not understanding — ten of them were introduced earlier the same day by the commit that created the detectors, which was verified by ancestry rather than assumed. Nobody claims a

targeted fix is an audit.

The eight introduced afterwards are the finding. Every one was written seventeen to eighteen days after the class was named, by the same author, in the same codebase, with the naming commit in the history and its tests passing. Four of the eight are in the two files that commit had itself edited.

Three explanations, in increasing order of discomfort:

- **The fix was applied to call sites, not to a shape.** It introduced no type, no helper, no construct that would make the next probe honest by default. A lesson that lives in prose must be recalled; a lesson that lives in a type cannot be forgotten.
- **The class is invisible at the moment of writing.** The failure branch is not where the author's attention is, and the value it returns is the natural thing to return. Writing the defect requires no error and no carelessness.
- **All eight were written in a two-day burst** — the densest period in the project. They cluster precisely where new surface was being created fastest. The rule was not rejected under pressure; it never came to mind.

Stated precisely, and narrowly:

Naming a defect class, documenting it in the code, and defending it with tests was not sufficient to prevent the same author from writing eight fresh instances of it in the same codebase within three weeks.

If that is the outcome under conditions this favourable — one author, a small codebase, the lesson written in the file being edited — it is unlikely to be better on a larger team.

12.5 The response, and what cannot be claimed

Three things changed, none of them more documentation: a **sweep** that is run rather than hoped for; **tests that assert the third value**, not only the negative; and **fixtures that can fail**.

We cannot report whether this worked. The observation window closes with the sweep on 22 September. A codebase swept once is not a codebase that stays swept, and on the evidence here the honest expectation is that instances will accumulate again — which is precisely what §12.4 found the first time.

13. The discipline of deletion

Three working, shipped modules were removed from this system within six weeks of being built. That is unusual enough to report, and the reasons are the same in all three cases.

Feature	Built	Removed	Lifetime
Warranty tracking	9 July (in the initial commit)	13 July	4 days
Sales, customers and orders	12 July	25 July	13 days
Repair logging	12 July	20 August	39 days

None was removed because it was broken. Each worked. Each was removed because **it modelled a business that does not exist**.

13.1 What each one got wrong

Warranty tracking was in the system from the very first commit, because an asset system "obviously" tracks warranties. Nobody used it, and the data needed to make it useful — per-device warranty terms from each manufacturer — was never going to be entered by hand. It was the smallest of the three: fields on the asset

rather than a module, which is why it could go in a single commit that dropped the columns with them. It was replaced with low-stock and out-of-stock alerts on consumables, which answer a question the warehouse actually asks: *are we about to run out of caddies?*

The sales module modelled an order book: customers, orders, line items, fulfilment. The business has no order book. Stock goes to trade buyers in bulk, and what needs recording is a **status transition** — this left, on this date, for this price. Every screen in the module asked for information nobody had.

It was replaced by **Sold as a status rather than a module**. Selling is terminal and locks the record; only an administrator can reverse it; and a return **reactivates whatever the device came from**, because otherwise a device comes back into a container that still says it has gone, and the stock counts quietly disagree with each other.

Repair logging assumed a repair workshop with per-device work logs. The warehouse grades devices and either sells them or breaks them for parts. Six weeks of non-use made that unambiguous.

The common failure is not over-engineering in the usual sense. Each module was a reasonable implementation of a real concept. The error was earlier: **the concept was imported from what inventory systems generally have, rather than derived from what this business actually does.**

13.2 Removing things safely

The third removal produced a house rule, which the first two were small enough not to need:

Drop the code first, drop the table second. Two commits, two deploys.

The code and the schema deploy separately. Dropping a table while code still references it is an outage; dropping the code first means the table sits unused for one deploy and then goes. This is the mirror image of the migration ordering hazard in §3.4 — in both cases the rule is that **code and schema must never be in a state where one expects something the other has not got.**

The second rule is about what survives a removal. The repairs module had accumulated **grading verdicts that were genuinely in use**, and those were preserved rather than going with the table. A module being wrong does not make everything in it wrong, and the work of separating the two is what makes deletion safe enough to do at all.

A related correction was made on the same principle: costing and invoicing had been attached to pallets, **because that is where they were first needed, not because they belong there**, and were unhooked. Misplaced attachment is a quieter version of the same mistake — the feature is right, the model is borrowed.

13.3 The argument

Adding a feature needs one person to think it is a good idea. Removing one needs somebody to say, out loud, that work already paid for should be thrown away — and to be right about it.

We think the deletions are why the system stayed buildable. The audit station, the erasure remediation and the inspection layer are all substantial pieces of work that arrived in August and September, into a codebase that had shed three modules it did not need. A codebase carrying warranty tracking, an order book and a repair workshop would have had three more sets of screens, migrations, permissions and tests to drag through every subsequent redesign — and each of them would have had to be kept working while the parts that mattered were being rebuilt.

The useful measure of a system under active development is not how much it contains. It is how much of what it contains is load-bearing.

13.4 The honest residue

The deletions were not total, and the paper should say so. The schema still carries customer, sales-order, order-line and invoice entities, and the API still has the corresponding modules. The user-facing sales workflow is gone and the Sold status replaced it, but the underlying tables were not all dropped.

This is left as it is rather than presented as finished. A reader inspecting the repository will find them, and a paper that claimed three clean removals while the residue sits in `apps/api/src` would be making exactly the kind of tidy, unsupported claim this project spent three months learning not to make.

14. Engineering practice

Three practices in this project are worth reporting because each exists to defend against a failure the previous sections keep describing: something that is wrong while appearing to be fine.

14.1 Design records

Every coherent piece of work ends with a dated record in the repository — 40 of them, covering 9 July to 24 September. Not one per commit: one per change, with its commits listed inside it.

The headings are **fixed**, because the value of the documents is that they can be read against each other:

Status (shipped / partial / blocked, with commit hashes) · *The problem* · *Why it matters* · *What was built* · **What was deliberately NOT built** · *How it was proved* · *Open questions*.

The fifth heading is the one that justifies the practice. A commit message says what changed; a diff says how. Neither recovers the alternative that looked equally good at the time and was rejected, and that is the information a person returning to the code six months later actually needs.

Three rules govern them, and the first is the point of the whole exercise:

1. **Honesty over tidiness.** A record saying "this was never tested on hardware" is worth more than one implying it was.
2. **No record is written before the work is done.** These are records, not plans.
3. **Name the failure modes** — especially the ones that survived review.

This paper is written from those records. That is the strongest practical argument for keeping them: a project documented only in commit messages cannot be written up afterwards without reconstructing the reasoning from memory, which is precisely the source this project has repeatedly found unreliable.

14.2 Testing something that erases drives

The audit station is shell and Python running against real hardware, and the thing it does most consequentially is destroy data. Testing it presents an obvious hazard.

The rule is absolute and enforced in the harnesses themselves: **a wipe test points at a temporary regular file, never a device node**. The harness refuses outright if its scratch directory is under `/dev`, and the stubbed tools refuse any invocation that names a device. A safety rule that exists only in the author's intention is one bad path expansion away from wiping a development machine.

The suites are bash and Python standard library **by rule** — no package installs, no database — for the same reason as §6.2's kiosk lesson: a dependency you have to install is one that will eventually not be there.

14.3 CI, and a list that stops covering things

Until the continuous-integration job existed, the station's tests ran only when somebody remembered to run them by hand on a Linux machine. A change to the engine that erases customer drives could merge with every one of them red.

One detail of that job is worth extracting as a general principle. The tests are discovered by a **glob, not a list**:

Every test file matching the pattern runs. A new test is covered the moment it is committed — because a hand-maintained list is exactly the thing that silently stops covering the newest test.

That is the paper's defect class applied to the build system. A list that has fallen behind reports the same green as a list that is complete.

The job deliberately covers what the hosting platforms' own builds cannot: the API typecheck, the full migration chain **from scratch against a real Postgres**, the seed, the newest migration's down-and-up, and a boot smoke test — because dependency-injection failures only surface at runtime, and a typecheck passes happily with a missing provider.

14.4 Two migration hazards

The first was described in §3.4: TypeORM selects every mapped column, so shipping an entity before its migration takes the table down. Closed by running migrations as a pre-deploy command on the newly built image, while the previous version still serves traffic.

The second is subtler and was caught by CI rather than in production. **A migration can work perfectly on the long-lived production database and destroy every fresh one**, because the whole chain runs in a single transaction: a statement that is harmless when applied to a table full of data can fail — or cascade — when applied to the chain building that table from nothing. Nobody runs the full chain from scratch in normal work. The only defence is a job that does it on every push.

The third, from §13.2, is the mirror image: on removal, drop the code first and the table second, because they deploy separately.

All three are instances of one rule, which is worth stating once:

Code and schema must never be in a state where one expects something the other does not have — in either direction, at any point in the deploy.

14.5 Where the practice is weak

Two gaps are real and are not presented as anything else.

The web application has no automated tests. Ninety-six test files cover the API and the station; the user interface is covered by the build, by manual use, and by the consistency passes of §10.5. Several of the defects those passes found would have been caught by tests that do not exist.

The station's tests are harnesses, not hardware. They stub the destructive tools and verify the logic around them. That is the right trade — the alternative is a lab of sacrificial drives — but it means the suites cannot catch a defect that lives in the gap between what a tool does in a stub and what it does on a real controller. Four of the defects in §12 were found by a technician on real hardware, and no test harness would have found them.

15. Results

This section reports what exists, what runs, and what has been proven — and is deliberate about the difference between those three.

15.1 What was built

Period	9 July – 24 September 2026 (78 days)
Commits	535
Code	~101,000 lines
Domain entities	23, across 62 schema migrations
Web pages	30
API controllers	23
Automated test files	96 (46 API, 50 audit station)
Design records	40
Engineers	1

Capability, by layer:

Layer	Delivered
Inventory	Three stock tiers, lot hierarchy, manifest reconciliation, pallets with a merge invariant, ownership scoping in six enforcement passes, append-only activity
Capture	Offline-first scanning (barcode and OCR) with a durable on-stick queue
Station	Bootable Secure Boot appliance, overlay layer, kiosk session, operator sign-in, sync by stick with version stamping
Inspection	Hardware profile, offline registry parsing, four management-state checks, per-drive health as a percentage, installed-OS capture
Erase	Method ladder with fallback reporting, per-drive verdicts, hidden-area checks, read-back after every erase
Certification	Signed, chained, tamper-evident certificates with optional public verification
Functional test	Seven technician-confirmed tests with three-valued verdicts
Back office	Eleven reporting slices, one-query operational dashboard, fail-closed permissions, WCAG 2.2 AA pass

15.2 What runs

The API, database and sync service run on one hosting platform; the web application on another; the audit station runs from USB sticks in the warehouse. All three are in production use by the business the system was built for.

We report no production telemetry, because none was collected. There are no figures here for machines processed, certificates issued, or time saved per device. That instrumentation was never built — the business needed the system to work before it needed the system measured — and inventing plausible numbers after the fact is exactly the failure this paper spends thirteen sections arguing against.

A reader evaluating whether to adopt any of this should know that the strongest operational claim we can make is that it is in daily use and has not been replaced.

15.3 What is proven, and how

The distinction that matters most is between what has been exercised on real hardware and what has only been reasoned about or tested in a harness.

Proven by	What
Real hardware, repeatedly	Bootling and full-screen operation across varied machines; hardware capture; drive health; the keyboard, speaker, camera and screen tests; offline registry reads on encrypted and domain-joined machines
Real hardware, once or twice	The overlay layer boot chain; the Autopilot artefact reads; the OOBE observation
Harness only	The wipe ladder's branch logic, the lock detectors' three-valued returns, the migration chain, the certificate signing and chaining
Reasoned, not run	The pre-installation-environment hash capture (§11.3, blocked)

Four of the 32 defects in §12 were found by a technician on real hardware and would not have been found by any harness. That ratio is the most useful calibration in this paper: **harnesses catch the logic, hardware catches the assumptions.**

15.4 The defect result

Restated here as a result rather than as an argument:

False-absence call sites catalogued	32 , across 15 fix commits
Failing towards the reassuring answer	32 / 32
Sitting on code paths that already had tests	18, of which caught by those tests: 0
Written after the class was named, documented and tested	8 , within 18 days

15.5 The negative results

Three things were attempted and did not work, and are reported as results because the field under-reports them:

Seven approaches to full-screen kiosk display failed before one succeeded, and six of the seven failed for the same reason: they needed a package that was not on the image (§6.2).

A layer rebuild could not be performed off the target machine, not for the obvious reason but because its safety gate would have passed *vacuously* in a container — reporting no conflict because it could not see, rather than because there was none (§6.6).

Autopilot registration cannot be determined offline. After substantial research this is reported as a boundary rather than a to-do (§11.2). The system answers the question three other ways, one of which is definitive, and none of which is the way the business originally asked for.

16. Limitations and threats to validity

16.1 One site, one business, one engineer

Everything here comes from a single ITAD business, a single codebase, and a single developer. The architecture is shaped by this business's actual processes — bulk trade sales rather than an order book, grading rather than repair — and §13 argues that fitting those processes closely was a virtue. The same argument means the design may not transfer to an ITAD operation that works differently.

The single-engineer fact cuts two ways in §12. It **strengthens** the recurrence finding — the author who wrote the defects was the author who named the class, so there is no question of the lesson failing to reach somebody. It **weakens** any generalisation to teams, where the mechanisms of transmission and forgetting are different and probably worse.

16.2 The defect catalogue classifies its own author's work

The 32 instances in §12 were identified, classified and counted by the person who wrote them. There is **no second rater**, and no inter-rater agreement to report. The inclusion criteria were fixed before the analysis and are published so the judgement can be inspected, but a reader who suspects the boundary was drawn to favour the result cannot presently be answered with anything except the dataset itself.

Re-rating ten randomly selected instances with an independent rater would address this, and is the single highest-value addition the work could receive.

Two narrower threats to the same catalogue:

- **It is a census, not a sample.** It reports every instance found in one codebase. The rates in §12 describe this system, and nothing licenses extrapolating them to software in general.
- **Provenance is imperfect.** The introducing commit was recovered by searching for a string the fix removed. Where that string post-dated the defect, the commit creating the enclosing function was used instead; four entries rest on that weaker method and are marked as such in the dataset.
- **Detection latency is an upper bound on time-to-detection, not a measure of exposure.** Station code does not reach a machine until the next USB sync, so a defect's presence in the repository is not the same as its presence on a bench.

16.3 What was never measured

No production telemetry exists (§15.2). There is no measured throughput, no error rate in the field, and no before-and-after comparison against the manual process the system replaced. Claims about the system's operational value rest on it being in continued daily use, which is weak evidence and is presented as such.

The deletion argument is an argument. §13.3 claims the three removals are why the system stayed buildable. Nothing measures that. The counterfactual — the same team carrying three unused modules through the September work — does not exist and cannot be constructed.

The remedy in §12.5 is unevaluated. The observation window closes with the sweep. Whether the sweep, the third-value tests and the corrected fixtures actually reduce the rate of new instances is unknown, and on the evidence of §12.4 the expectation should be modest.

16.4 Gaps in the system itself

Gap	Status
The web application has no automated tests	Real; several defects found by manual passes would have been caught
Station tests are harnesses, never real drives	Deliberate trade; means hardware-level assumptions go untested (§15.3)
Hardware-hash capture before wipe (K3)	Blocked — needs a pre-installation environment and a bench session
Per-user data isolation beyond manager scoping	Deferred
Sales, customer and invoice tables	Residue of a removed module (§13.4)

16.5 Two decisions that are the owner's, not ours

Both are recorded here rather than silently defaulted, because each determines what a document the business issues is allowed to say:

1. **Should a machine whose drive is hidden behind a RAID controller be certifiable at all?** The system can report that it could not see the drive. Whether that machine may then be sold with a certificate is a commercial and legal judgement.

2. **Should a lock discovered after issue reach a certificate already in a buyer's hands?** The first-boot observation of §11.3 can establish a registration *after* the machine has been certified and sold. There is currently no mechanism for amending an issued certificate, and whether there should be is not an engineering question.

16.6 Scope of the knowability framework

§11's four tiers were derived from one vendor's device-management ecosystem. K1, K2 and K3 are mechanical and should transfer. **K4 is the interesting tier and has exactly one worked example.** Whether other classes of withheld state behave the same way — in particular whether they can always be converted to an observable event the way Autopilot's first boot can — is untested, and the framework should be read as a proposal rather than a validated taxonomy.

17. Lessons for building evidence-producing systems

The thesis, restated as the constraint it actually is:

Every output of this system is a claim about a machine the system does not control. Its entire value is how honestly it separates what it has established from what it has merely assumed.

What follows is what that cost to learn, in the order a team would need it.

17.1 Before writing a probe

Classify the claim by knowability tier (§11). Is it measured, recovered from an artefact, perishable, or withheld? The tier determines which kind of effort will work: code, a parser, a process change, or none. Writing a cleverer probe for a perishable fact is wasted work; writing one for a withheld fact produces a guess with the appearance of a measurement.

Ask whether the failure value equals the negative value. If the probe returns empty, zero, false or absent both when it looked and found nothing *and* when it could not look, any assertion built on it is unearned. This single question, applied at the keyboard, is the cheapest defence available and it found 20 of the 32 defects in §12.

17.2 While designing the vocabulary

Three values, in three places. *Present*, *absent* and **not established** — and the third has to be sayable in **the type**, **the stored field**, and **the sentence a person reads**. Implementing the first two and not the third produces a system that knows it could not check and prints *not detected* anyway. We did exactly that more than once.

Decide which way the third value leans, per domain. In lock detection, an unestablished result must not read as *safe*, so it promotes the device to unverified (§7.4). In functional testing, an unestablished result must not read as *broken*, so it is attention rather than failure (§9.2). These look like opposite rules and are the same rule: **the third value must not collapse into whichever of the other two is convenient.**

Never cache a failure. An answer may be memoised; an inability to answer may not. One dead call settled a hardware question for an entire session because the failure was cached alongside the successes.

17.3 While writing tests

A fixture that supplies a working probe tests nothing about failure. Eighteen of the 32 defects sat on tested code paths and the tests caught none. Catching this class needs a fixture where **the probe fails and the subject is adverse** — two independently unlikely conditions that no author constructs without deciding to.

When an honesty check breaks a test, fix the fixture. This happened four times, and every time the fixture modelled a command that could not fail. A test that must be weakened to accommodate an honesty check was asserting the bug.

Discover tests with a glob, not a list (§14.3). A hand-maintained list that has fallen behind reports the same green as a complete one.

17.4 While shipping

A dependency you have to install is a dependency you do not have (§6.2). Seven approaches to one problem, six of them failing for this reason. On any appliance, live-boot image, or constrained target, the packages already present are the only ones that exist.

Code and schema must never disagree in either direction (§14.4). Migrate before the code that needs the column; drop the code before the table it uses. Both failures look like an outage and neither looks like a schema problem.

Instrument what the software is doing, not only what it found. The kiosk work became tractable only when the status bar showed *which of five launch paths had run*. Before that, every failure looked identical. The same applies to a network check that says which address it reached and a restore that shows it is alive.

A summary and its drill-down must be computed from the same predicate (§10.3). Otherwise the page contradicts itself at the moment a user trusts it most — and only when somebody clicks.

17.5 About the organisation, not the code

Naming a defect class is not a control. This is the finding we least expected and most want others to have. A commit that named the class, explained it in the affected functions and added tests covering a third of its own diff did not stop the same author writing eight fresh instances within three weeks (§12.4).

The reason is that the fix was applied to call sites rather than to a shape. **A lesson that lives in prose must be recalled; a lesson that lives in a type cannot be forgotten.** So:

Instead of	Do
Documenting the rule	Making the wrong thing hard to express — a type, a helper, a return shape
Trusting it will be noticed	Running an explicit, repeatable search for the shape
Fixing the instances you found	Reading every site in the component that probes

Look where code touches state it does not control. 28 of 32 instances were in the component that boots a customer's machine; 23 were in two files. A team adopting nothing else can act on that: find the component that probes, read every site where a read result becomes a reported finding, and apply §17.1's second question.

Removal is a feature. Three shipped modules were deleted because they modelled a business that did not exist (§13). The useful measure of a system under active development is not how much it contains but how much of what it contains is load-bearing — and deletions need a safe pattern, because the dangerous part is the schema, not the code.

17.6 The one we would tell someone first

Of everything above, the finding with the widest application is the smallest:

In a system that looks for problems, a probe that fails returns the value that means "no problem". So every failure resolves towards the reassuring answer, silently, and gets certified.

Thirty-two instances. Not one false alarm. The asymmetry is not a property of this codebase; it is a property of what empty, zero and false mean in a system whose job is to find things wrong.

18. Related work

This work sits at the intersection of four literatures, and is not squarely inside any of them. That is the honest description of a systems paper about an industrial problem: the components are well studied, the combination is not.

18.1 Media sanitisation and asset disposition

The technical basis for erasure is standardised. NIST's media-sanitization guidance establishes the distinction between clearing, purging and destroying media, and per-medium expectations for each — the framework §8's method ladder implements. Industry certification schemes for asset disposition specify process and audit requirements for refurbishers.

What that literature does not address, and what §8 reports, is the **software engineering** of producing a certificate that is true: the per-drive verdict, the read-back that must not treat an unreadable drive as verified, the hidden areas an erase does not reach, and the case where a certificate already issued turns out to be wrong. Standards specify what must be achieved. They do not specify how a program that cannot read a drive should describe what it did.

18.2 Offline-first and local-first systems

The architecture of §5 — local database, background reconciliation, eventual convergence — is the local-first pattern, and the literature on it is concerned mainly with data convergence and user data ownership. Our contribution to that discussion is narrow and operational: in a system where the client's writes are **evidence**, the queue's failure modes matter more than its merge semantics, and three of the four faults we hit in the first week were failures to *report* rather than failures to converge.

18.3 Dependability and error handling

The fault / error / failure taxonomy gives the vocabulary for silent failure and for a **fail-silent** component, which stops rather than emitting a wrong result. The class in §12 is the complement: the component neither stops nor signals, and emits a result that is wrong in a consistent direction.

Empirical studies of error handling in distributed systems establish that error-handling code is disproportionately defective and under-tested. §12 is narrower and orthogonal: we characterise what the erroneous handler *produces*, and show that in a reporting system its output has a direction — always towards the reassuring answer.

Two adjacent traditions are worth naming because both already solved a version of this problem. **Instrumentation and control practice** distinguishes "no signal" from "signal reads zero" and invests in validity flags to keep them apart; that discipline has not transferred to general-purpose software that probes an environment. And **type systems and database theory** have long separated *absent* from *unknown*. §12 treats that distinction as the remedy rather than the subject, and contributes evidence that knowing about it is not sufficient to apply it.

18.4 Alarm design, inverted

Work on alarm fatigue in clinical monitoring documents the cost of false positives, and the design responses to them. The class described here is the mirror image: a failure mode that generates **no alarms at all**, and is therefore invisible to every process built to respond to them. We are not aware of a literature on the inverse problem — systems whose failures are silent because their failure value is indistinguishable from good news —

and would welcome being told one exists.

18.5 What we did not find

We searched for a prior naming of the defect class in §12 and did not find one, but that search is not exhaustive and its result is provisional. **If a prior naming exists, this paper's framing changes:** the contribution narrows from naming and measuring to measuring alone, and §12.4's recurrence result becomes the sole novel finding. We would rather state that conditionally than discover it in review.

We also found no prior treatment of what §11 calls knowability tiers as an explicit engineering framework, though the underlying distinctions are commonplace in practice. §16.6 states plainly that the tier which makes the framework interesting rests on a single worked example.

19. Conclusion

An ITAD business sells second-hand computers and issues documents saying what they are and that the data on them is gone. Those documents are claims about machines nobody in the business controls, made once, briefly, by a technician, about hardware that is then sold. Nothing in the normal operation of such a business tells you when one of them is wrong.

We built the whole system for that job in 78 days: an offline-first inventory platform, a bootable audit appliance that examines and erases a customer's machine without ever starting its operating system, an inspection layer that reads a powered-off Windows out of its own registry, an erasure engine whose certificates are signed and chained, and a functional-test module that treats a technician's answer as the measurement it is. Part II describes each layer and what was rejected in building it.

Three findings generalise beyond the business.

Claims have knowability tiers, and the tier decides what effort will work. Measured, recovered from an artefact, perishable, or withheld by a third party. Code answers the first two. The third needs a process change upstream of whatever destroys the fact. The fourth needs to be reported as unanswered — and the most valuable question this system could answer, whether a machine is registered to somebody else's provisioning tenant, turned out to sit there. We built three partial answers, one of them definitive, by abandoning deduction and observing the moment the machine itself finds out.

In a system that looks for problems, every failed probe fails towards the reassuring answer. Empty, zero, false and absent are the values that mean *no problem*. We catalogued 32 call sites where a failed observation was reported as a confirmed negative, and all 32 pointed the same way. Eighteen sat on code paths that already had tests, and those tests caught none of them, because a fixture is written from the author's model of the path working.

Naming the defect class was not enough. A commit named it, documented it in the affected functions, and defended the fix with tests worth a third of its own diff. Eight fresh instances were written within the next three weeks, by the same author, in the same files. The lesson had been recorded and was not recalled — because a rule that lives in prose must be remembered, and the moment it is needed is the moment attention is on making something work. The remedy is not more emphasis. It is to move the rule into a type, into fixtures that can fail, and into a search that is run rather than hoped for.

What we cannot report is whether that worked. The observation window closes with the sweep, and a codebase swept once is not a codebase that stays swept. We report the response rather than a cure, for the same reason the rest of this paper distinguishes what was established from what was assumed — which is the only discipline that made any of it trustworthy.

The system is in daily use. Its most valuable property is not any single capability described here. It is that when it cannot establish something, it says so.